

Raport

From subject:

**“Biologically inspired artificial
intelligence”**

**“Analysis of Artificial Intelligence models
used for financial data”**

Mikołaj Mieszczok

1. Introduction

This project aims to predict the best time to sell a stock after a trade made by a U.S. congress member is made public. Public financial disclosures by lawmakers have become a topic of high interest, and platforms like capitoltrades.com provide this data. By combining this information with historical stock prices retrieved via yfinance, and applying machine learning models like Random Forest, LightGBM, and LSTM, the goal is to build a system that can learn from past trade disclosures and recommend an optimal future sell date.

2. Analysis

(a) Approach

1. Classification Approach

- **Description:** Predict a class (e.g., Day 5, Day 10, Day 15) as the optimal sell day.
- **Pros:** Clear, straightforward; easier to evaluate.
- **Cons:** Less granular; potentially ignores the subtleties of daily price changes.

2. Regression Approach

- **Description:** Predict the return for each day and choose the max.
- **Pros:** Captures continuous outcomes.
- **Cons:** Sensitive to outliers; harder to evaluate directly.

3. Time-Series Forecasting (LSTM)

- **Description:** Learn temporal dependencies to forecast price movements.
- **Pros:** Captures trends and patterns in stock data.
- **Cons:** Requires large datasets and is computationally expensive.

I decided not to focus on 1 specific approach but rather try different ones and compare them in order to find the most profitable model. That would of course require a little bit more work, but at the same time it would guarantee that I tested more possibilities.

(b) Dataset

There is only one dataset that contains the trades of US politicians, but it does not satisfy the needs of my project since it does not list the dates of the trades, but only what a politician bought in their lifetime, so I had to make my own using a web scrapper. The columns in the dataset represent:

The politicians first and last name, the stock he bought, the amount of days that passed since he reported the trade, the amount of days after the report after which the stock was worth the most, provided it was more than 5% of the original value, how the stock behaved in the 42 days proceeding the report.

Jonathan Jackson	CRM.US	28	26	1	0.987711	0.975964	0.9625	0.98253	0.988042	0.989729
Tony Wied	TTD.US	4	25	1	1.044483	1.048653	1.0298	0.739314	0.717202	0.701129
Marjorie Taylor Greene	AMZN.US	3	26	1	0.991613	0.997505	0.991226	0.984301	0.985806	0.984172
Marjorie Taylor Greene	AAPL.US	4	26	1	0.988059	0.982162	0.995074	1.01965	1.038329	1.05081
Marjorie Taylor Greene	CAT.US	3	35	1	0.989384	0.979661	0.966825	0.954827	0.965741	0.962681
Marjorie Taylor Greene	DELL.US	3	36	1	0.998419	1.025298	1.023531	1.033203	1.032738	1.083612
Marjorie Taylor Greene	FDX.US	3	25	1	0.992971	0.998687	0.996601	1.012822	1.021047	1.038658
Marjorie Taylor Greene	FDX.US	4	25	1	0.992971	0.998687	0.996601	1.012822	1.021047	1.038658
Marjorie Taylor Greene	PI.US	3	26	1	0.979273	0.94981	0.968863	0.976392	0.962915	0.976206
Marjorie Taylor Greene	JPM.US	3	28	1	0.995961	0.974718	0.988495	0.993616	1.000361	0.998161
Marjorie Taylor Greene	LRCX.US	3	36	1	1.004327	0.998437	0.97704	0.980647	0.993509	1.020676
Marjorie Taylor Greene	LULU.US	3	30	1	0.961269	0.961438	0.948834	0.965881	0.95021	0.882986
Marjorie Taylor Greene	NKE.US	3	35	1	0.982993	1.000142	1.000567	1.026786	1.03699	1.055839

(c) Tools, frameworks and libraries

The first library used was BeautifulSoup, a library that simplifies extracting data from HTML, it enabled me to fill corresponding arrays with correct data:

```
url = url_start + str(x)
page = urlopen(url)
html_bytes = page.read()
html = html_bytes.decode("utf-8")
soup = BeautifulSoup(html, features='html.parser')
politician_links = soup.find_all(name='a', class_='text-tx-interactive')
stock_links = soup.find_all(name='span', class_='q-field issuer-ticker')
filed_after_links = soup.find_all(name='span', class_='reporting-gap-tier--1', 'reporting-gap-tier--2', 'reporting-gap-tier--3')
whether_buy_links = soup.find_all(name='span', class_='q-field tx-type tx-type--buy', 'q-field tx-type tx-type--sell')
date_filed_links = soup.find_all(name='div', class_='text-size-3 font-medium')
amount_bought_links = soup.find_all(name='span', class_='mt-1 text-size-2 text-tx-dimmer hover:text-foreground')
year_bought_links = soup.find_all(name='div', class_='text-size-2 text-tx-dimmer')
```

The second important library was yfinance which made getting a price of a stock on a certain day or through a period of time very simple:

```

for row in range(1, 10):
    try:
        print(row)
        row = str(row)
        not_filtered = yf.Ticker(str(ws["B" + row].value).split('.')[0]).history(start = str((ws["D" + row].value).split(' ')[0], end = str((ws["E" + row].value).split(' ')[0], interval = "1d")
        #print(not_filtered)
        if not_filtered.empty:
            continue
        filtered = not_filtered["Open"]
        for col, price in enumerate(filtered):
            worksheet.write(columns[col] + row, price)
    except Exception as e:
        print(f"Error at row {row}: {e}")

```

I also used `wlsxwriter` and `openpyxl` to read and write to excel files:

```

workbook = xlsxwriter.Workbook("C:/Users/Mikołaj/Desktop/Engineer/FIRSTEXCEL.xlsx")
worksheet = workbook.add_worksheet()
write_data_into_column(worksheet, res, column: "A")
write_data_into_column(worksheet, stocks, column: "B")
write_data_into_column(worksheet, filed_after, column: "C")
write_data_into_column(worksheet, sixty_days_ago, column: "D")
write_data_into_column(worksheet, todays_date, column: "E")
workbook.close()

workbook = xlsxwriter.Workbook("C:/Users/Mikołaj/Desktop/Engineer/SECONDEXCEL.xlsx")
worksheet = workbook.add_worksheet()
wb = openpyxl.load_workbook("C:/Users/Mikołaj/Desktop/Engineer/FIRSTEXCEL.xlsx")
ws = wb.active

```

For Random Forest and LGBM I used `scikit-learn` library to simplify the usage and automatically get parameters like loss and accuracy

```

val_preds_proba = model.predict_proba(X_val)
val_preds = np.argmax(val_preds_proba, axis=1)

accuracy = accuracy_score(y_val, val_preds)
loss = log_loss(y_val, val_preds_proba)

```

3. Internal and external specification of the software solution

A) Classes/objects/methods/scripts/functions

The project is written in 3 separate scripts:

- `DiamondHands`(webscrapper), with functions that change data in order for it to be passed to `yfinance`.

```

def filter_politicians(array):
    for x in range(len(array)):
        if(x%2 != 0):
            array[x]="0"

```

```
def write_data_into_column(worksheet_temp, arr, column): 5 usages
    worksheet_temp.set_column(str(column) + ":" + str(column), 20)
    for i, data in enumerate(arr):
        worksheet_temp.write(str(column) + str(i+1), str(data))
```

```
def remove_even_indexes(arr):
    return [arr[i] for i in range(len(arr)) if i % 2 != 0]
```

```
def remove_odd_indexes(arr): 1 usage
    return [arr[i] for i in range(len(arr)) if i % 2 == 0]
```

```
def convert_date_format(date_str):
    month_map = {
        "Jan": "01", "Feb": "02", "Mar": "03", "Apr": "04", "May": "05", "Jun": "06", "Jul": "07", "Aug": "08", "Sept": "09", "Oct": "10", "Nov": "11", "Dec": "12"
    }
    day, month = date_str.split()
    return f"{day}.{month_map[month]}."
```

```
def merge_dates(date_str, date_filed_str):
    return str(date_filed_str) + str(date_str)
```

```
def remove_marital_status(year_bought): 1 usage
    indices_to_keep = [
        i for i in range(len(year_bought)) if i%3==0
    ]
    year_bought[:] = [year_bought[i] for i in indices_to_keep]
```

```
def remove_all_non_buy(politicians, stocks, filed_after, whether_buy, date_filled, year_bought):
    indices_to_keep = [
        i for i in range(len(whether_buy)) if whether_buy[i] != 0 and stocks[i] != 'N/A'
    ]

    # Update lists in place
    politicians[:] = [politicians[i] for i in indices_to_keep]
    stocks[:] = [stocks[i] for i in indices_to_keep]
    filed_after[:] = [filed_after[i] for i in indices_to_keep]
    whether_buy[:] = [whether_buy[i] for i in indices_to_keep]
    date_filled[:] = [date_filled[i] for i in indices_to_keep]
    year_bought[:] = [year_bought[i] for i in indices_to_keep]
```

```
def remove_items(test_list, item): 4 usages
    res = [i for i in test_list if i != item]
    return res
```

```
def trim_array(arr, index): 3 usages
    return arr[:index]
```

- StockDataToExcel – taking data from FirstExcel created by DiamondHands adding created data by yfinance from SecondExcel and passing it all into a ready-to-be-assessed ThirdExcel


```

for row in range(1, 10):
    try:
        print(row)
        row = str(row)
        not_filtered = yf.Ticker(str(ws["B" + row].value).split(".")[0]).history(start = str(((ws["D" + row].value)).split(" ")[0]))
        #print(not_filtered)
        if not_filtered.empty:
            continue
        filtered = not_filtered["Open"]
        for col, price in enumerate(filtered):
            worksheet.write(columns[col] + row, price)
    except Exception as e:
        print(f"Error at row {row}: {e}")

for y in range(len(columns)-1):
    if ws_second[str(columns[y + 1] + str(x))].value is not None:
        worksheet.write(str(columns[y]) + str(x), str(ws_second[str(columns[y + 1] + str(x))].value / division))

```

- LSTM/RandomForest/LGBM

The last files are all focused on training and testing the chosen model.

```

X = train_df.drop(columns=["D"])
y = train_df["D"].astype(int)
y_cat = to_categorical(y)
num_classes = y_cat.shape[1]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_test = test_df.drop(columns=["D"], errors='ignore')
X_test_scaled = scaler.transform(X_test)
X_train, X_val, y_train, y_val = train_test_split(X_scaled, y_cat, test_size=0.2, random_state=42, stratify=y)
X_train = X_train.reshape((X_train.shape[0], 1, X_train.shape[1]))
X_val = X_val.reshape((X_val.shape[0], 1, X_val.shape[1]))
X_test_resaped = X_test_scaled.reshape((X_test_scaled.shape[0], 1, X_test_scaled.shape[1]))
y_int = np.argmax(y_train, axis=1)
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(y_int),
    y=y_int
)
class_weights = dict(enumerate(class_weights))
model = Sequential()
model.add(LSTM(128, return_sequences=True, input_shape=(1, X_train.shape[2])))
model.add(Dropout(0.4))
model.add(LSTM(64))
model.add(Dropout(0.3))
model.add(Dense(64, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
optimizer = Adam(learning_rate=1e-4)
model.compile(loss='categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
early_stop = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
history = model.fit(
    X_train, y_train,
    epochs=200,
    batch_size=32,
    validation_data=(X_val, y_val),
    class_weight=class_weights,
    callbacks=[early_stop],
    verbose=1
)

```

B)user interface / GUI/console

The project does not have an interface as of yet, since I don't plan on distributing the software as of now. The path to excels is set and not changable outside of code.

4. Experiments

To test out individuals models performance I decided to cut out a 500 trade sample from the 13000 trade database and test how much profit I could generate listening to the models suggestions using a slightly modified version of StockDataToExcel

Michael McCaul	GE.US	29	1	0.990698	0.984061	0.99274	1.020703	0.99603	0.998775	0.98186	1.042271	1.055633	1.119171
Michael McCaul	NFLX.US	35	1	1.005992	1.001666	1.007771	0.985308	0.974109	0.978916	0.981719	1.0145	1.001218	1.019594
Michael McCaul	GE.US	38	1	0.990698	0.984061	0.99274	1.020703	0.99603	0.998775	0.98186	1.042271	1.055633	1.119171
Michael McCaul	SAFRY.US	39	1	1.008581	1.01734	1.025742	1.021809	1.028245	0.982124	0.986056	1.006793	0.981409	1.009474
Michael McCaul	GE.US	32	1	0.990698	0.984061	0.99274	1.020703	0.99603	0.998775	0.98186	1.042271	1.055633	1.119171
Michael McCaul	TPR.US	44	1	0.989671	0.968592	0.964165	0.98946	1.013491	0.975759	0.970067	0.963533	0.911046	0.907251
Michael McCaul	QQQ.US	37	1	0.995958	1.002133	1.002448	0.99872	0.999147	0.988167	0.985652	1.0022	0.980757	0.991153
Michael McCaul	VOO.US	37	1	0.995541	0.998312	0.99925	1.002084	1.003272	0.992623	0.991602	1.002772	0.985371	0.994332
Michael McCaul	VTI.US	37	1	0.995795	0.998495	0.999769	1.002353	1.00436	0.99348	0.990509	1.001775	0.984259	0.994637
Michael McCaul	VTI.US	37	1	0.995795	0.998495	0.999769	1.002353	1.00436	0.99348	0.990509	1.001775	0.984259	0.994637
Marjorie Taylor Gr	AMD.US	0	1	0.975679	1.013148	1.015518	1.012753	1.016308	1.012753	1.001862	1.032222	0.948592	0.949156
Chuck Fleischmanr	FLJP.US	20	1	1.001958	1.007833	1.015992	1.021214	1.007833	1.013708	1.015013	1.009465	0.997389	0.989556
Chuck Fleischmanr	XMHQ.US	20	1	0.988186	0.998781	1.018096	1.025691	1.022316	1.02391	1.024004	1.030849	1.03413	1.023535
Chuck Fleischmanr	SHY.US	20	1	1.000613	1.000859	1.002577	1.003067	1.002945	1.002576	1.003558	1.00319	1.002943	1.00122
Chuck Fleischmanr	IGSB.US	20	1	1.000588	1.001567	1.003526	1.004505	1.00431	1.002938	1.004114	1.004309	1.004407	1.001459
Chuck Fleischmanr	HYBB.US	20	1	0.999129	1.002396	1.007231	1.00893	1.004574	1.005445	1.004792	1.007406	1.006749	1.001713
Chuck Fleischmanr	CALF.US	20	1	0.991169	0.999369	1.017287	1.025922	1.016865	1.019182	1.01855	1.030977	1.038559	1.017708
Chuck Fleischmanr	MDYV.US	20	1	0.99287	0.999863	1.021253	1.025915	1.017962	1.021527	1.021253	1.03757	1.044289	1.024544
Chuck Fleischmanr	GLDM.US	20	1	0.999766	0.997428	1.020973	1.008417	1.007248	1.014964	1.013327	1.024316	1.044891	1.047229
Shri Thanedar	GOOGL.US	43	1	0.996972	0.995895	1.005787	1.003432	1.00895	1.010834	1.017294	1.015073	1.013996	1.032905
Shri Thanedar	GOOGL.US	43	1	0.996972	0.995895	1.005787	1.003432	1.00895	1.010834	1.017294	1.015073	1.013996	1.032905
Shri Thanedar	GOOGL.US	43	1	0.996972	0.995895	1.005787	1.003432	1.00895	1.010834	1.017294	1.015073	1.013996	1.032905
Shri Thanedar	AAPL.US	43	1	0.992994	1.000854	1.00843	0.978299	0.971521	0.968275	0.97061	0.978242	0.975053	0.963035
Shri Thanedar	AAPL.US	43	1	0.992994	1.000854	1.00843	0.978299	0.971521	0.968275	0.97061	0.978242	0.975053	0.963035

Here are the results:

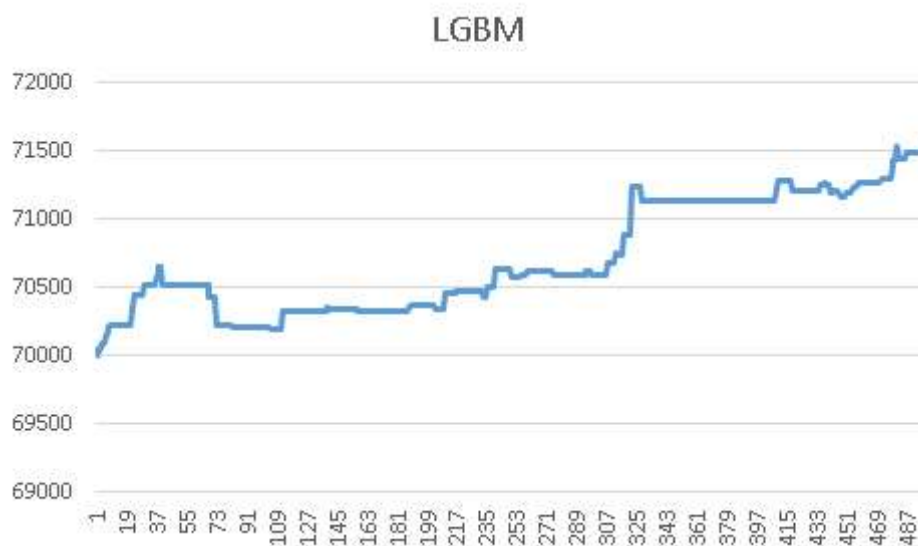
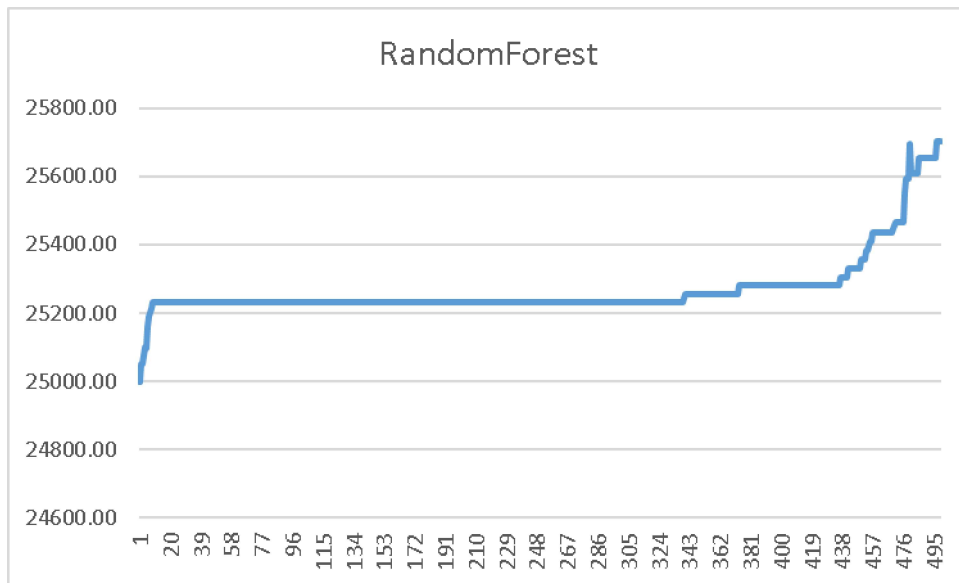
	RF	LGBM	LSTM 200	LSTM 400	LSTM 600
Profit	653.039	1491.4	-62.556	4811.983	313.3388954
nr of trades	25	70	19	28	35
% Profit	2.61216	2.13057	-0.3292	17.18565	0.895253987
avr trade profit	3.0565	2.2667	-0.368	18.50763	0.949511804

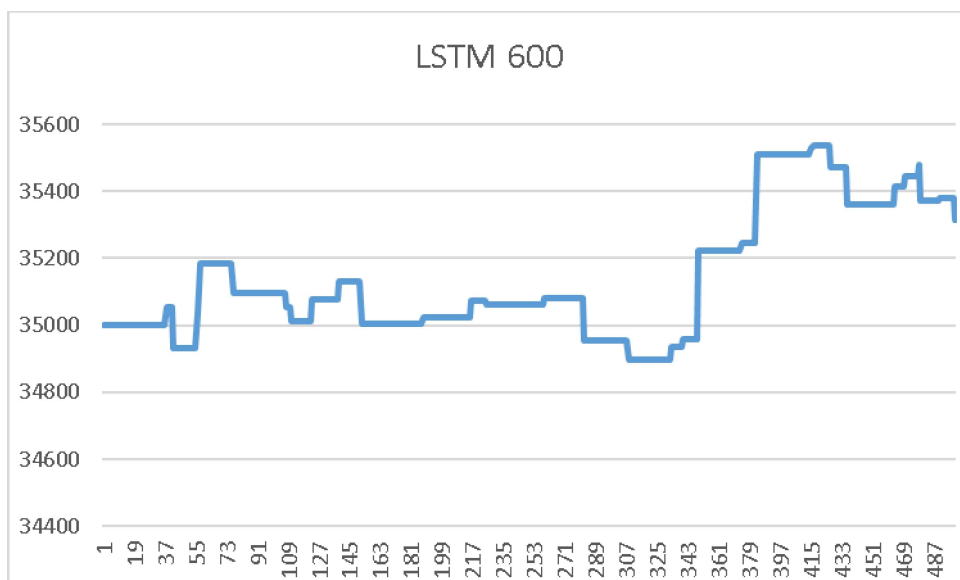
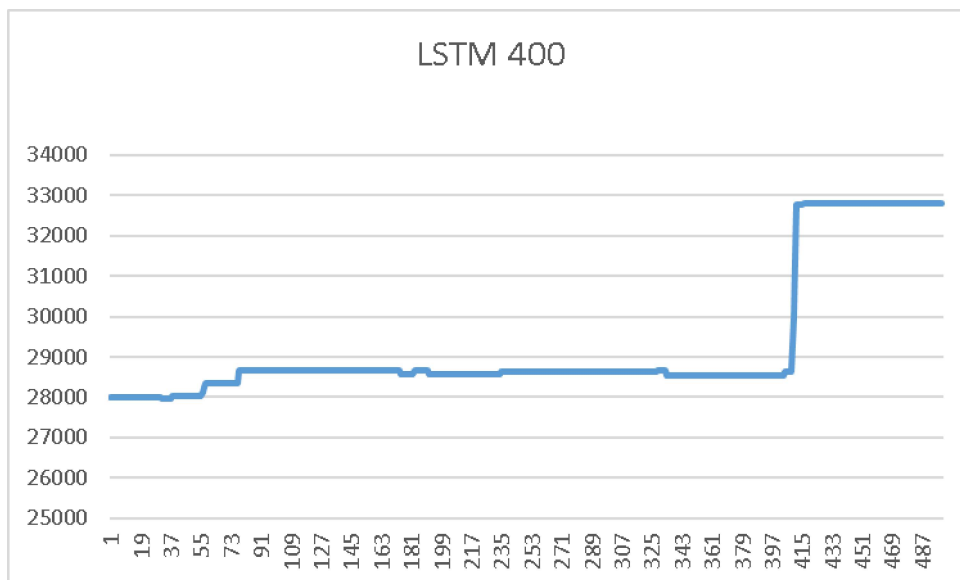
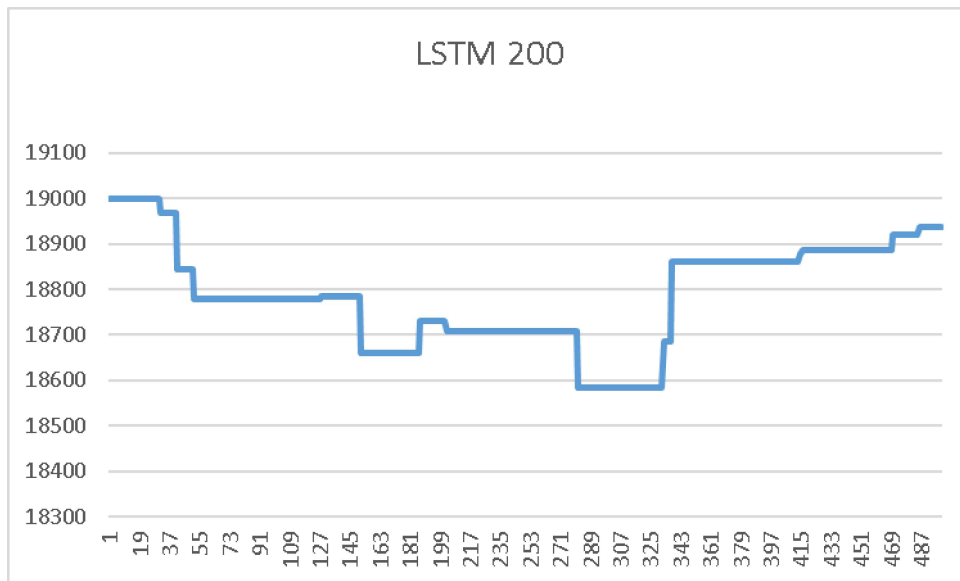
Profit is calculated by assigning every trade 1000 PLN, and later subtracting 1000 from the price it was sold at. % Profit calculated using this formula:

$$\% Profit = \left(\frac{Profit + 1000 \cdot nr\ of\ trades}{1000 \cdot nr\ of\ trades} - 1 \right) \cdot 100$$

And average trade profit is the average of all profits of all trades made by a specific model

Here how it looks on a plot. X axis represents the nr of trades and the Y axis the account state:





The LSTM's poor performance is most likely caused by the unsuitability of the dataset to the needs of LSTM, it requires sequential data and my dataset has fields that are not sequential. I'll tinker with the dataset making it all about the data from the past and use the politicians data only as signals to analyse a stock. As for LGBM and RandomForest they performed quite well. Accumulating a profit of about 3% (since the date at which I bought + amount of days after which I sold does not exceed the date at which I bought of some trades the profit is actually a bit higher than 2,61%) is more than can be said for any risk free forms of investing on a 40 day period. If we used the formula for the n-th element of a geometric sequence, our estimated % gain in the span of the year would be 26.7%.

The profitability is fairly represented using the accuracy of models, RF which scored the highest also generated the most profit:

```
Validation Accuracy: 0.3937  
Validation Log Loss: 2.0582
```

Next was LGBM:

```
Validation Accuracy: 0.3893  
Validation Log Loss: 3.0160
```

And lastly LSTM:

```
val_accuracy: 0.3508
```

5. Summary, overall conclusions, possible improvements, future work

The project taught me a lot: working with a web-scraper, a bit of python, working with different AI models. The performance of models is substantial enough for me to continue pursuing the project, I even selected it as my engineering thesis. The project still needs a lot of work:

- Full incorporate and test LSTM
- Find the most optimal solution
- Automatize everything
- Train model on sold stocks as well
- Incorporate news as means to influence model's decisions
- Capitoltrades.com also has a field disclosing how much of the stock a politician bought. In the future I'll check for correlations between that amount and how well the stock performed to adjust the amount of invest, instead of blindly investing 1000 PLN into every one.

6. References

<https://ranaroussi.github.io/yfinance/>

https://xlsxwriter.readthedocs.io/getting_started.html

<https://openpyxl.readthedocs.io/en/stable/>

<https://scikit-learn.org/stable/>

<https://www.tensorflow.org/guide?hl=pl>

<https://www.capitoltrades.com/trades>

7. Link to github

<https://github.com/MikolajMieszczok/GetRichQuickLegal>